# SOF Diagnostic Protocol

*Realization-Relative Reports for Strict SOF Realizations and Diagnostic Analogues*

WuJun Chen

Independent Researcher

2026

*This paper is Paper XII of the RIME program. It consumes the typed interfaces of Papers VIII–XI and owns the single-report SOFRS protocol and its epistemic boundary.*

## Abstract

**Problem.** Paper X establishes capability-sound report compilation, but a source system need not determine a unique sectorization, observable extraction, analogue mapping, or report profile. The remaining question is therefore what an assembled report represents about its source.
**Approach.** This paper defines the capability-aware **SOF Report Specification (SOFRS) v2.1**. An admitted adapter $\eta$ supplies a Capability Manifest $M_\eta$ and validated Typed SOF IR $I_\eta$; Paper X compiles them under $P_X$, and SOFRS faithfully assembles the result under $P_A$:

$$\mathcal{O}_{\eta,P_X} = \mathrm{Compile}_{v1}(M_\eta, I_\eta, P_X), \qquad \mathcal{R}^{\mathrm{view}} = \mathrm{Assemble}_{v2}(\eta, M_\eta, I_\eta, P_X, \mathcal{O}_{\eta,P_X}, P_A).$$

**Results.** Report Relativity identifies a family indexed by source snapshot, adapter, profiles, and version closure rather than one canonical report attached to the source. Assembly Faithfulness preserves every normative compiler item exactly once. Report Non-Intervention states that compilation, assembly, serialization, and validation act on report artifacts rather than on the reported source state, while the Adapter Adequacy Boundary keeps protocol conformance distinct from scientific adequacy. A source-addressed migration preserves nine frozen v1 reports as diagnostic analogues, records four bounded reconstruction assessments, and replaces 118 legacy cutoff sentinels with typed states.
**Implications.** A SOFRS report is a capability-gated, realization-relative epistemic artifact. Missing capabilities are not converted to zero findings, infinity, failed bridges, or nearby carriers; claim status, claim target, and certificate class remain independently typed.

---

## Notation Table

| Symbol | Meaning |
|---|---|
| $S_\sigma$ | source system at the pinned source snapshot $\sigma$ |
| $\eta$ | admitted domain adapter selecting a realization and its capabilities |
| $M_\eta$ | Capability Manifest supplied by the adapter |
| $I_\eta$ | validated Typed SOF IR supplied to the Paper X compiler |
| $P_X$ | Paper X Compiler Report Profile selecting compiler modules and items |
| $P_A$ | Paper XII Assembly Profile governing faithful report rendering |
| $\mathcal{O}_{\eta,P_X}$ | Paper X `CompilerOutput` produced by `Compile_v1` |
| $\mathcal{R}^{\mathrm{norm}}$ | normative report core assembled from compiler output |
| $\mathcal{R}^{\mathrm{view}}$ | assembled SOFRS report view for the declared source, adapter, profiles, and version closure |
| $\mathcal{A}$ | serialized `.sofreport` artifact with canonical provenance and validation bindings |
| $\mathfrak{A}_{\mathrm{art}}$ | state space of report-pipeline artifacts, distinct from source state |
| $\pi_S$ | projection from joint source/artifact protocol state to source state |
| $v_N, v_A$ | normative and assembly version closures |
| `external_basis_registry` | claim-scoped registry of source-addressed external constraint packages |
| `claim_target` | typed object of a reader-facing claim, such as an external object or protocol conformance |
| `certificate_class` | typed scope of a certificate: Object, Protocol, or Migration/Assembly |
| `provenance.kind` | mutually exclusive `native_generation` or `migration` provenance variant |

The table records the protocol's principal mathematical and machine-contract objects. Complete field inventories and status mappings remain in the SOFRS schema and its appendices.

---

# 1 Introduction

Papers VIII–XI supply the typed objects, dynamic fields, capability-aware compiler contracts, and wall morphology consumed here [3, 2, 1, 4]. This paper assumes the Paper X compiler result and asks the remaining epistemic and protocol-level question:

> **What does a report assembled from capability-sound compiler output represent about its source system?**

The answer is not a canonical map $S \mapsto \mathcal{R}$. A source may admit multiple scientifically defensible sectorizations, observable extractions, analogue mappings, thresholds, cutoffs, or Compiler Profiles. The protocol therefore retains the declared adapter, compiler-profile, and assembly-profile choices

$$S \xrightarrow{\mathrm{Adapter}_\eta} (M_\eta, I_\eta) \xrightarrow{\mathrm{Compile}_{v1}(\cdot, P_X)} \mathcal{O}_{\eta,P_X} \xrightarrow{\mathrm{Assemble}_{v2}(\cdot, P_A)} \mathcal{R}^{\mathrm{view}}_{\sigma,\eta,P_X,P_A,v_N,v_A}(S).$$

Different $\eta$, $P_X$, or $P_A$ may therefore yield different valid reports. By contrast, invalidity arises from failed admission, evidence, policy, or promotion rules.

This paper has three governing contributions:

1. **Report Relativity** separates semantic report equality, canonical artifact equality, aligned comparability, and realization invariance.
2. **Adapter Adequacy Boundary** separates compiler soundness from scientific adequacy of the selected realization.
3. **Versioned Reporting Protocol** defines SOFRS v2.1, with Assembly Faithfulness as its central executable invariant.

---

## 2 Related Work and Novelty Boundary

**Program interfaces.** This paper consumes the typed object, deformation, compiler, Registry, and wall-record interfaces of Papers VIII–XI without re-owning them [3, 2, 1, 4]. Alignment and pairwise comparison belong to Paper XIII; interpretation belongs to Paper XIV.

**Reporting and provenance precedents.** Structured testing, profiling, static-analysis, model-card, and algorithmic-audit reports motivate explicit scope and limitation records [9, 12]. SOFRS specializes that discipline to carrier-qualified claims, typed unavailability, certificate classes, and digest-closed provenance.

**Diagnostic domain contexts.** Attention, routing, and diffusion studies motivate bounded examples of sectorization-like decompositions [13, 5, 11, 8, 6, 7]; they do not supply a general explainability or load-balancing theorem here.

**Novelty boundary.** The contribution is a realization-relative, carrier-qualified, digest-closed reporting protocol with faithful assembly and typed negative boundaries. Protocol conformance neither re-proves Paper X nor establishes adapter adequacy.

---

## 3 Inherited Paper X Compiler Contracts

This paper uses four Paper X results as inherited infrastructure rather than claiming them again. The compiler theorem invoked here is bound specifically to **Capability Manifest v1.0**, **Typed SOF IR v1.0**, and **Compiler Report Profile v1.0**; a future major contract version requires a new soundness statement.

| Object or result | Owner | Paper XII role |
|---|---|---|
| Capability Manifest v1.0 | Paper X | instantiate source capabilities and unavailable carriers |
| Typed SOF IR v1.0 | Paper X | populate typed objects, findings, evidence, and audited derivations |
| Compiler Report Profile v1.0 ($P_X$) | Paper X | select admissible compiler modules and items |
| Capability-Sound Report Compilation | Paper X | assume compiler soundness under the versioned contract hypotheses |
| Report Relativity | Paper XII | state how adapter and profile choices index valid reports |
| Adapter Adequacy Boundary | Paper XII | separate compiler soundness from scientific adequacy and source fidelity |
| Versioned Reporting Protocol (SOFRS v2.1) | Paper XII | define strict/analogue report structure, present graceful degradation, and govern migration, deployment, and failure boundaries |
| SOFRS Assembly Profile v2.0 ($P_A$) | Paper XII | render one fixed `CompilerOutput` without changing its normative items |

The Manifest contains no result, the IR selects no presentation, and the Profile creates no evidence. Paper X supplies the capability- and carrier-qualified `CompilerOutput`; this paper defines how that fixed output is assembled, serialized, migrated, and presented as one report.

---

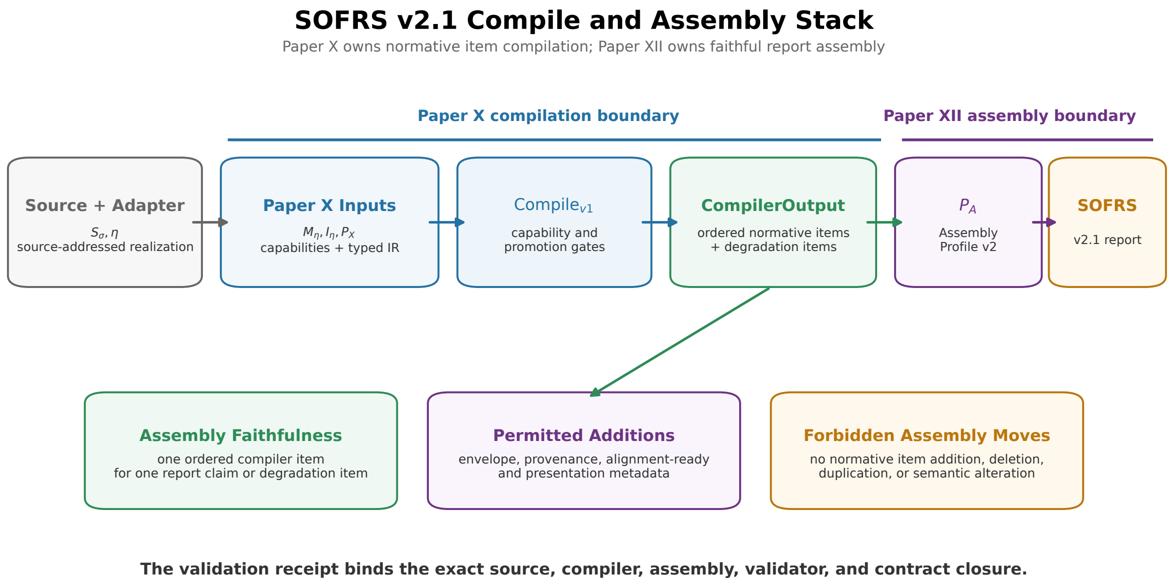


**Figure 1.** SOFRS v2.1 compile and assembly stack. Paper X compiles capability-gated normative items from a Manifest, Typed SOF IR, and Compiler Profile. SOFRS assembles the fixed CompilerOutput under an Assembly Profile without adding, deleting, duplicating, or altering a normative item.

# 4 Admission Modes and Axes

## 4.1 Admission Modes

Each Manifest and IR declares exactly one `record_kind`.

| Record kind | Minimum admission data | Permitted conclusion |
|---|---|---|
| `strict_sof` | finite complex $V$, complete marked projectors $\{Q_i\}$, labelled alphabet $Y$, structural certificate, and declared conventions | carrier-qualified SOF findings and compatible theorem or computational instances |
| `diagnostic_analogue` | source and evaluator provenance, stable descriptors, an analogue mapping, and a negative SOF boundary | structurally analogous diagnostics only |

The strict core is

$$\mathcal{F}_{\text{op}} = (V, \{Q_i\}_{i\in I}, Y).$$

Declaring this core does not automatically declare route, word, closure, Lie/Hall, or deformation capabilities. Conversely, a diagnostic analogue does not become strict by accumulating observations. Promotion requires a new adapter construction and structural validation of explicit $(V, Q, Y)$ data.

A strict record may carry a `proxy_diagnostic` side module. The record remains strict because its core is strict; the proxy remains a proxy because it is carrier-qualified. One record cannot simultaneously declare `diagnostic_analogue`.

## 4.2 Orthogonal Admission and Claim Axes

The protocol separates five questions.

| Axis | Controlled values | Question answered |
|---|---|---|
| record kind | `strict_sof`, `diagnostic_analogue` | what mathematical kind of report is admitted? |
| source-map status | native; adapter-derived; migrated | how were the reported objects or descriptors obtained? |
| evidence status | result state plus reader-facing claim status | what was established, certified, observed, left open, or unavailable? |
| claim target | external mathematical object; empirical domain system; representation interface; protocol conformance; migration consistency | what kind of object does the statement concern? |
| certificate class | object; protocol conformance; migration/assembly | what does a finite certificate actually certify? |

The source-map vocabulary records provenance of construction rather than scientific quality. Consequently, `heuristic` is a pre-admission adapter-development label, not a valid status in an assembled SOFRS report; by definition, it indicates that no admissible strict or analogue record has yet been constructed. Every admitted source mapping must instead identify its adapter, construction, justification, limitations, and source artifacts.

A result state records what happened: `DECLARED`, `ESTABLISHED`, `CERTIFIED`, `OBSERVED`, `UNREACHED_AT_CUTOFF`, `NOT_APPLICABLE`, or `NOT_DECLARED`. The independent reader-facing claim status uses exactly four levels: Theorem, Computational Certificate, Computational Observation, and Research Program.

These pairings are constrained. `ESTABLISHED` accompanies a theorem, `CERTIFIED` accompanies a reproducible finite certificate, and `OBSERVED` accompanies a bounded computational observation. `UNREACHED_AT_CUTOFF` requires an explicit cutoff policy and never denotes exact infinity. `NOT_DECLARED` and `NOT_APPLICABLE` carry no positive claim status.

Evidence strength alone is insufficient. A `Computational Certificate` in the inherited Paper X vocabulary must therefore be refined by a certificate class:

| Certificate class | Certified object | Does not establish |
|---|---|---|
| Object Certificate | a finite matrix, graph, depth, trajectory, or other source-level fact independently recomputed from source artifacts | generalization, causal interpretation, or adapter optimality |
| Protocol Certificate | satisfaction of a declared schema, type, policy, provenance, or compiler contract | truth of the represented scientific claim |
| Migration Certificate | preservation of declared invariants across compilation, assembly, or version conversion | recomputation of the source experiment |

Here, Protocol Certificate abbreviates Protocol Conformance Certificate, and Migration Certificate abbreviates Migration/Assembly Certificate. The machine fields are `claim_target`, `certificate_class`, and `classification_source`. Paper X v1 does not contain `claim_target`; SOFRS must therefore identify whether the classification came from a domain adapter, Assembly Profile, or migration adapter. It must not present a report-layer classification as inherited compiler content.

The protocol uses the following compatibility matrix:

| Claim target | Certificate class | Authorized classification sources |
|---|---|---|
| external mathematical object | Object | compiler IR, domain adapter, independent validator, or external evaluator |
| empirical domain system | Object, or none for an observation | domain adapter, independent validator, or external evaluator |
| representation interface | Protocol, or none for a bounded adapter observation | compiler IR, assembly profile or validator, domain adapter, or migration adapter |
| protocol conformance | Protocol | compiler IR, assembly profile or validator, or independent validator |
| migration consistency | Migration/Assembly | migration adapter, assembly validator, or independent validator |

Here, an independent validator requires a declared implementation, owner, or execution-environment basis; digest difference alone does not establish independence.

In particular, an Assembly Profile cannot classify an external mathematical object as true. The validator also enforces the exact pairings `ESTABLISHED`/Theorem, `CERTIFIED`/Computational Certificate, and `OBSERVED`/Computational Observation.

### 4.3 Claim-Scoped External Basis

Protocol conformance is not an external scientific check. SOFRS therefore associates each report with a finite registry of named external-basis packages at four distinct semantic levels: source identity, object-level recomputation, realization/structure validation, and domain semantic adequacy. Each claim cites only the packages and constraints used for that claim; satisfaction is never inherited merely because another claim cites the same report.

Each package is satisfied, partial, not assessed, or not applicable, and every satisfied package binds source-addressed evidence. A report may therefore be protocol-valid while the object-level or domain-level basis of a particular claim remains unresolved. In the migration controls, for example, the frozen source identity is satisfied while independent object recomputation and semantic adequacy remain unassessed.

The registry constrains certificate admission rather than authorizing new scientific claims. An Object Certificate requires a satisfied object-level basis with independently checkable evidence; strict-SOF admission requires a satisfied structure-level basis. Without those conditions, the result remains protocol/representation evidence or carries an unresolved external basis. The complete machine mapping, identifiers, and status vocabulary are given in Appendix A.

This is an admissibility condition for certificate labeling, not a theorem of scientific adequacy. A domain adapter remains responsible for choosing a meaningful source realization and baseline. SOFRS records that responsibility, the evidence supplied for it, and the negative boundary when the evidence is absent.

Record kind, source-map status, and evidence status are independently declared. A strict object may support only a bounded observation, while an analogue may carry a reproducible finite certificate about its own evaluator outputs. What the analogue cannot do is instantiate a strict-SOF theorem.

**Principle (Admission Separation).** Forgetting a strict carrier may produce a separate analogue export, but that operation changes the record kind and must be performed by an explicit adapter. It is not an inclusion or promotion inside one hierarchy.

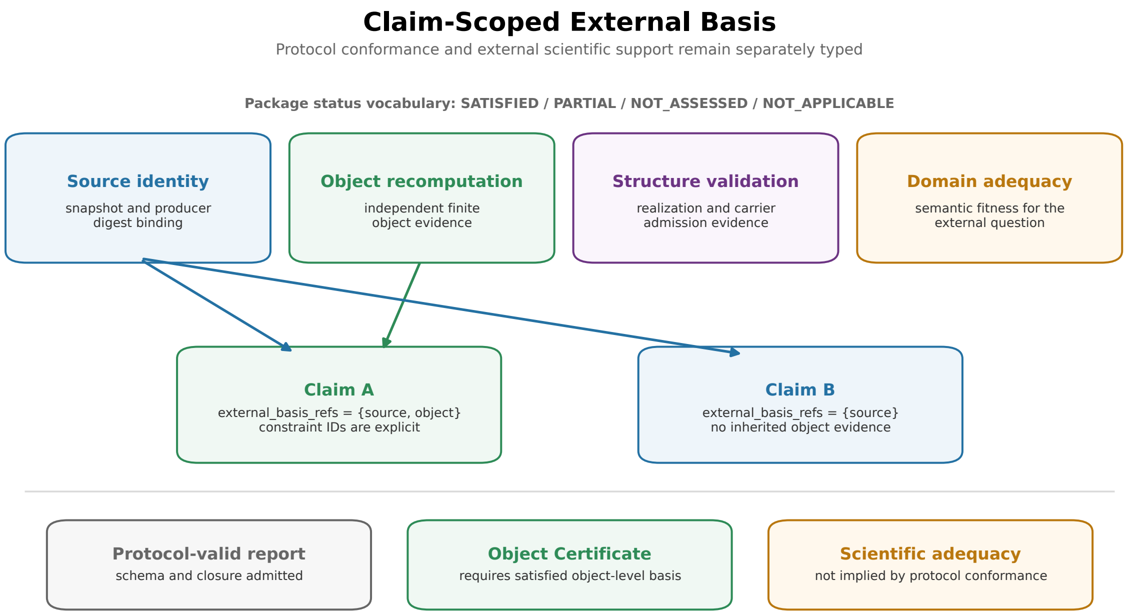


**Figure 2.** Claim-scoped epistemic boundary. External-basis packages separately record source identity, object recomputation, structure validation, and domain adequacy. Each claim cites only its own basis packages; protocol conformance does not supply missing external scientific support.

## 5 Report Relativity

**Definition 5.1** (SOFRS v2.1 Report Protocol)**.**

Let $S_\sigma$ be a canonically identified source snapshot, let $\eta$ be an admitted domain adapter producing a schema-valid Capability Manifest $M_\eta$ and validated Typed SOF IR $I_\eta$, let $P_X$ be an applicable Paper X Compiler Profile, and let $P_A$ be a compatible Paper XII Assembly Profile. Paper X first produces

$$\mathcal{O}_{\eta,P_X} = \mathrm{Compile}_{v1}(M_\eta, I_\eta, P_X).$$

The normative report core is

$$\mathcal{R}^{\mathrm{norm}}_{\sigma,\eta,P_X,v_N} = \mathrm{NormCore}_{v_N}(\sigma, \eta, M_\eta, I_\eta, P_X, \mathcal{O}_{\eta,P_X}),$$

where $v_N$ fixes the normative compiler, claim, degradation, and report-core contracts. The assembled report view is

$$\mathcal{R}^{\mathrm{view}}_{\sigma,\eta,P_X,P_A,v_N,v_A} = \mathrm{Assemble}_{v2}(\eta, M_\eta, I_\eta, P_X, \mathcal{O}_{\eta,P_X}, P_A; v_N, v_A),$$

where $v_A$ fixes the assembly contract. Finally, a serialization contract $v_S$ produces

$$\mathcal{A}_{\sigma,\eta,P_X,P_A,v_N,v_A,v_S} = \mathrm{Serialize}_{v_S}(\mathcal{R}^{\mathrm{view}}_{\sigma,\eta,P_X,P_A,v_N,v_A}).$$

Assembly is defined mathematically on typed objects, whereas the artifact protocol accepts immutable references to the source snapshot, adapter implementation, Manifest, IR, Compiler Profile, `CompilerOutput`, and Assembly Profile, each with its version and digest. Because a serialized report contains both normative and non-normative fields, the Assembly Profile declares which view fields remain non-normative.

Write

$$\mathrm{NormItems}(\mathcal{R}) = \mathrm{ClaimItems}(\mathcal{R}) \sqcup \mathrm{DegradationItems}(\mathcal{R}).$$

Valid assembly requires a type- and identity-preserving bijection

$$\alpha : \mathcal{O}_{\eta,P_X} \xrightarrow{\cong} \mathrm{NormItems}(\mathcal{R}^{\mathrm{norm}}_{\sigma,\eta,P_X,v_N}).$$

Each rendered item retains a `source_output_item_id`. Claim items remain claim items; degradation items remain degradation items. Adapter or application failure modes are separate report metadata and never substitute for compiler degradation items.

**Protocol Invariant 1 (Assembly Faithfulness).** For every validated input closure, $\mathrm{Assemble}_{v2}$ neither adds, deletes, duplicates, nor changes a normative `CompilerOutput` item. It adds only the versioned envelope, provenance, alignment-ready metadata, migration metadata, and presentation fields licensed by $P_A$.

**Verification argument.** The assembly rules traverse the ordered `CompilerOutput.items` array once. A `ClaimItem_v1` produces exactly one rendered claim item and a `DegradationItem_v1` produces exactly one rendered degradation item, both with the source item identity and kind retained. No other rule constructs a normative item. The validator recomputes $\mathrm{Compile}_{v1}$, reconstructs the report with $\mathrm{Assemble}_{v2}$, checks object equality, and verifies that the item-binding relation is a bijection.

The source-to-report chain is therefore

$$S \xrightarrow{\mathrm{Adapter}_\eta} (M_\eta, I_\eta) \xrightarrow{\mathrm{Compile}_{v1}(\cdot,P_X)} \mathcal{O}_{\eta,P_X} \xrightarrow{\mathrm{Assemble}_{v2}(\cdot,P_A)} \mathcal{R}^{\mathrm{view}}_{\sigma,\eta,P_X,P_A,v_N,v_A}.$$

**Protocol Invariant 2 (Report Non-Intervention).** For a fixed source snapshot, admitted realization, compiler inputs, Assembly Profile, and versioned contract closure, compilation, assembly, serialization, and report validation produce or verify report-layer artifacts only. They do not define or induce a transition of the reported source object.

Equivalently, lift the report pipeline to a joint protocol state consisting of the pinned source and an artifact state $a \in \mathfrak{A}_{\mathrm{art}}$:

$$\mathrm{ReportPipe} : (S_\sigma, a) \longmapsto (S_\sigma, a').$$

Then its source projection is invariant,

$$\pi_S(\mathrm{ReportPipe}(S_\sigma, a)) = S_\sigma.$$

**Verification argument.** The normative type signatures return `CompilerOutput`, report objects, serialized artifacts, or validation receipts. None has a source-state codomain, and the protocol defines no arrow $\mathcal{R} \to S'$ or $\mathcal{A} \to S'$. This proves protocol non-intervention. It does not prove implementation purity: a nonconforming program could perform an external side effect while constructing an artifact. Read-only source access, workspace confinement, and side-effect isolation are separate runtime controls.

Here $\eta$ may construct either a strict realization

$$\mathcal{F}_\eta = (V_\eta, \{Q_i^\eta\}, Y_\eta)$$

or a diagnostic analogue with an explicit source mapping and negative strict boundary. The choice records retained data, labels, extraction rules, and source-to-report decisions.

**Proposition 5.2** (Report Relativity).

*SOFRS v2.1 defines a family of normative report cores $\{\mathcal{R}^{\mathrm{norm}}_{\sigma,\eta,P_X,v_N}\}$ and assembled views $\{\mathcal{R}^{\mathrm{view}}_{\sigma,\eta,P_X,P_A,v_N,v_A}\}$, not a canonical map from an unversioned source $S$ to one report. A canonical normative report requires canonical identification of the source snapshot, admitted adapter,* *Compiler Profile, and normative version closure together with deterministic normative construction. A canonical view additionally requires an Assembly Profile and assembly version. Canonical artifact identity also requires a canonical serialization contract and encoding.*

***Argument.*** *Definition 5.1 makes the assembled report a function of the declared input closure*

$$(\sigma, \eta, M_\eta, I_\eta, P_X, P_A, \mathcal{O}_{\eta,P_X}, v_N, v_A)$$

*rather than of $S$ alone. Distinct admitted adapters may select different sectorizations, alphabets, analogue mappings, or policies, and distinct Compiler Profiles may select different claim and degradation items from the same validated IR. Assembly Profiles may change only non-normative presentation. The Paper X contracts make each compiler output sound relative to its declarations; Protocol Invariant 1 preserves that output but does not canonically select the source snapshot, adapter, profiles, or version closure. Thus a unique report does not follow from the source system alone.*

*Semantic report equality is equality of normative report cores. Equivalently, it quotients assembled views by fields declared non-normative by the Assembly Profile. Two views may therefore differ in licensed presentation metadata while remaining semantically equal. Canonical artifact equality is stronger: it requires equal canonical bytes, or equivalently equal digests under the fixed serialization contract.*

*The proposition separates three non-implications:*

$$\textit{same source} \not\Rightarrow \textit{same realization},$$

$$\textit{same realization} \not\Rightarrow \textit{same Compiler Profile},$$

$$\textit{different reports} \not\Rightarrow \textit{one report is erroneous}.$$

*Only a contract violation, unsupported promotion, failed evidence condition, or false source declaration makes a report **protocol-inadmissible**. Different valid choices alone do not.*

*The six layers must not be identified:*

| Layer | Object | Role |
|---|---|---|
| Source | $S$ | the underlying physical, algebraic, computational, or behavioral system |
| Realized object | $\mathcal{F}_\eta$ | the typed strict SOF realization or declared diagnostic analogue represented in $(M_\eta, I_\eta)$ |
| Compiler output | $\mathcal{O}_{\eta,P_X}$ | Paper X claim and degradation items selected by $P_X$ |
| Normative report core | $\mathcal{R}^{\mathrm{norm}}_{\sigma,\eta,P_X,v_N}$ | compiler-derived claims and degradation items with normative envelope fields |
| Assembled report view | $\mathcal{R}^{\mathrm{view}}_{\sigma,\eta,P_X,P_A,v_N,v_A}$ | the faithfully assembled single-system view, including licensed non-normative presentation fields |
| Serialized artifact | $\mathcal{A}_{\sigma,\eta,P_X,P_A,v_N,v_A,v_S}$ | a versioned `.sofreport` encoding with digests and provenance |

The SOF Report is a capability-gated, realization-relative protocol object, distinct from the source, realized object, and serialized file. Its boundary is:

> A single-system SOFRS report, absent an external reference, specification, or comparison contract, describes the declared realization but does not by itself establish comparative conformance or correctness.

Formal proofs and certificates can establish claims within the declared realization, but not conformance to another model, implementation, or specification without an external comparison datum.

The conceptual division is concise:

**Sectorization determines the ontology of the report; observable families determine its epistemology.**

For a strict record, sectorization declares the marked parts and the labelled alphabet declares operative witnesses. For an analogue, descriptors and their mapping declare what is being compared without asserting projector semantics. The Manifest and IR declare conventions, policies, provenance, and claim boundaries. Altering any of these may alter the report without altering the underlying source system.

For two admissible realizations of the same source,

$$\mathcal{R}_1 = \mathrm{Assemble}_{v2}(\eta_1, M_{\eta_1}, I_{\eta_1}, P_{X,1}, \mathcal{O}_{\eta_1,P_{X,1}}, P_{A,1}),$$
$$\mathcal{R}_2 = \mathrm{Assemble}_{v2}(\eta_2, M_{\eta_2}, I_{\eta_2}, P_{X,2}, \mathcal{O}_{\eta_2,P_{X,2}}, P_{A,2}).$$

one may have $\mathcal{R}_1 \neq \mathcal{R}_2$ even though the source $S$ is unchanged. The discrepancy may come from $\eta_1 \neq \eta_2$, from different Compiler Profiles or version closures, or from non-normative presentation choices. A report difference is therefore not, by itself, a system difference or a defect.

Four notions should be kept separate:

1. **Semantic report equality:** two normative report cores are equal after parsing under the same contract and canonical semantic normalization. Fields declared non-normative by the Assembly Profile are quotiented out.

2. **Canonical artifact equality:** two canonical serializations have identical bytes, equivalently the same digest under the declared algorithm. Equal JSON values with different key order or encoding need not have equal raw bytes before canonicalization.

3. **Aligned report comparability:** sectors, observables, depth semantics, and normalization are related by explicit maps. This is the object of Paper XIII.

4. **Realization invariance:** a statement survives a declared class of admissible realizations. This is a stronger theorem-level property and must be proved rather than assumed.

## 5.1 Alignment-Ready Metadata

SOFRS v2.1 does not perform pairwise alignment, but it must preserve enough typed provenance for a later alignment protocol to determine whether alignment is possible. Each report therefore exposes:

1. adapter, Compiler Profile, and Assembly Profile identifiers and versions;

2. sector labels, provenance, and available ranks or dimensions;

3. observable labels and operative semantics;

4. declared carrier kinds;

5. word, Hall, direction, depth, and projector-letter conventions when applicable;

6. cutoff, saturation, threshold, norm, and trajectory policies when applicable;

7. comparison keys or external identifiers;

8. source-artifact digests.

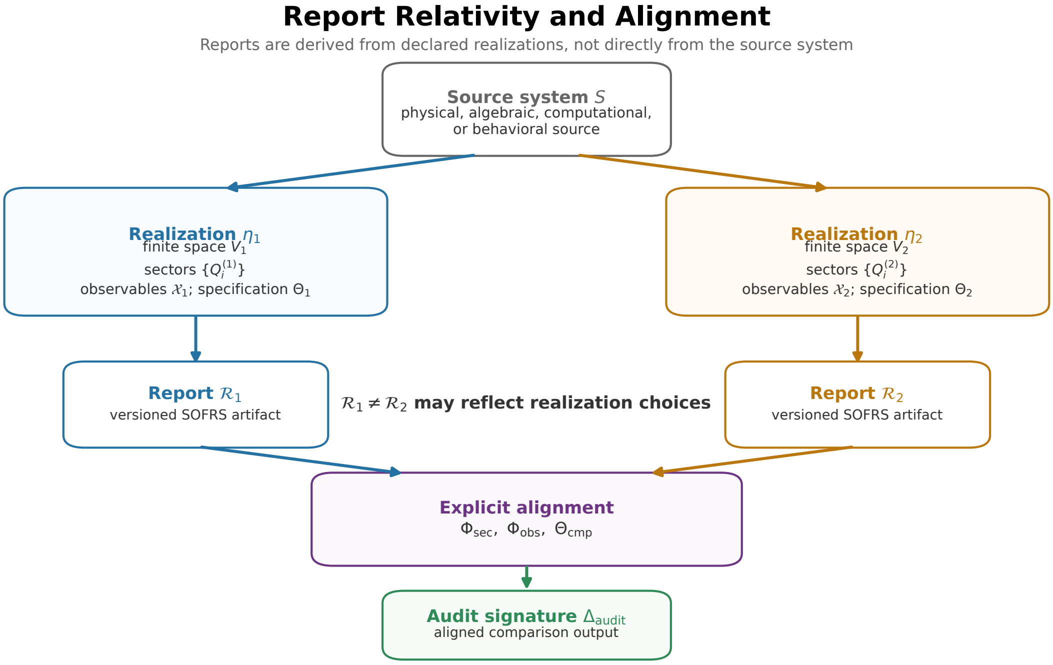


**Figure 3.** Report Relativity and Alignment. One source system may admit multiple declared realizations and therefore multiple versioned SOF Reports. A report difference is not by itself a source-system difference: sector, observable, and comparison alignment are required before an Audit Signature is formed.

These fields are **alignment-ready metadata**, not an alignment map. Paper XIII owns the actual sector and observable correspondences $(\Phi_{\text{sec}}, \Phi_{\text{obs}})$ and the comparison specification $\Theta$. Missing or incompatible metadata may make a later comparison `INCOMPARABLE`; it must not be repaired by label guessing.

---

## 6 Adapter Adequacy Boundary

Paper X proves conditional compiler soundness: given valid declarations and evidence, compilation does not manufacture a claim or cross a forbidden carrier boundary. It does not prove that an adapter is scientifically adequate for its source domain.

**Principle 1 (Adapter Adequacy Boundary).** Compiler soundness is internal to the declared contracts. It does not imply that the selected strict realization or analogue mapping is scientifically adequate for the source question. Adequacy remains an explicit domain-adapter obligation; it cannot be inferred from schema validity alone.

An adapter must therefore justify:

1. **source fidelity:** which source data are retained, transformed, or discarded;
2. **sectorization justification:** why the marked parts or analogue descriptors are meaningful for the stated question;
3. **observable adequacy:** what the alphabet or evaluator can and cannot detect;
4. **policy justification:** why thresholds, norms, cutoffs, sampling, and truncation are appropriate;

5. **negative-boundary completeness:** which nearby interpretations and promotions are explicitly excluded;

6. **provenance:** which inputs, code, environments, and artifacts support the record.

| Owner | Responsibility |
|---|---|
| Paper X compiler contracts | internal admission, capability, policy, evidence, derivation, and promotion soundness |
| Paper XII protocol | report organization, unavailable-state presentation, provenance, and migration discipline |
| Domain adapter | scientific adequacy of the source realization or analogue mapping |

A schema-valid adapter may still be scientifically weak or misleading. SOFRS makes that choice inspectable; it cannot replace domain judgment.

---

# 7 Versioned SOFRS v2.1 Reporting Protocol

The protocol assembles one validated Paper X compiler output into:

The assembly contains a versioned header, admission and capability state, typed findings, evidence and artifact references, explicit limitations, and a canonical `.sofreport.json` serialization.

The operational pipeline has a thin common waist:

$$\begin{aligned}\text{source} &\to \text{domain adapter}\\ &\to (M_\eta, I_\eta)\\ &\to \text{typed validation}\\ &\to \mathcal{O}_{\eta,P_X} = \text{Compile}_{v1}(M_\eta, I_\eta, P_X)\\ &\to \text{Assemble}_{v2}(\cdot, P_A)\\ &\to \mathcal{R}^{\text{view}}_{\sigma,\eta,P_X,P_A,v_N,v_A}\\ &\to \mathcal{A}_{\sigma,\eta,P_X,P_A,v_N,v_A,v_S}.\end{aligned}$$

The operational order is source registration, adapter and capability declaration, layered validation and IR construction, Compiler Profile selection, and faithful SOFRS assembly. Missing capabilities remain unavailable and are not filled from nearby carriers.

Strict profiles may select SOF Basic, Associative, Closure, Lie/Hall, and Dynamic modules; none is universally required. An application with no Lie family emits no Lie-depth conclusion, rather than a zero result. An analogue profile instead selects descriptor provenance, analogue mapping, measured values, and the negative strict boundary.

## 7.1 Protocol Stack Boundary

SOFRS is the descriptive layer of the protocol stack. Its unit is one named source realization, run, snapshot, aggregate diagnostic, or admissible trajectory:

$$\text{typed IR} \xrightarrow{\text{Compile}_{v1}} \text{CompilerOutput} \xrightarrow{\text{Assemble}_{\text{SOFRS}}} \mathcal{R}_{\text{SOF}}.$$

It does not align two independent reports, compute a cross-report difference, interpret that difference as an action consequence, or select a policy.

| Paper | Input and operation | Artifact |
|---|---|---|
| XII | assemble and serialize one validated `CompilerOutput` | `.sofreport.json` |
| XIII | align two reports and compute an Audit Signature | `.sofaudit` |
| XIV (downstream) | interpret signature coordinates under declared context and policy; emit bounded candidates | `.sofaction` |

An internal wall or repair finding remains descriptive. It does not prescribe `repair`, `monitor`, `preserve`, `contain`, or `validate` actions.

---

# 8 Protocol Instantiation Patterns

White-box and behavioral access describe the source interface, but they do not determine admission. A white-box analysis can fail strict admission, while a finite adapter derived from internal data can pass it.

| Source access | Possible v2 admission | Boundary |
|---|---|---|
| internal matrices, activations, transitions, or graph operators | strict if complete $(V, Q, Y)$ data pass validation; otherwise analogue | internal access alone does not establish strictness |
| external protocols, task classes, response classes, or interface states | normally diagnostic analogue | probe labels are not projector sectors |
| mixed source | either path according to the adapter output | record kind is determined by declared objects, not branding |

The methodological point remains that **SOF is an observable framework, not a weight framework.** Weights are neither necessary nor sufficient for strict admission. A black-box report may provide valuable structural, behavioral, failure, and repair diagnostics, but it cannot reconstruct hidden operators or instantiate a theorem about $\mathsf{SOF}_{\mathrm{str}}$.

**Principle 2 (Analogue Boundary).** A diagnostic analogue may describe a structure that resembles support, bridge, repair, or wall behavior. Those descriptors remain on the analogue carrier until an explicit realization or bridge theorem supplies the missing strict objects.

## 8.1 Quantum Probe Example

The focused quantum controls (Artifacts B17–B18) exhibit report relativity on one quantum gate-family comparison.

| Realization | Sectorization | Observable family | Observed T sensitivity |
|---|---|---|---|
| gate-log | computational-basis projectors | skew-Hermitian gate logarithms | none; the Clifford and Universal $R_1^{\mathrm{Lie}}/R_2^{\mathrm{Lie}}/D_{\mathrm{Lie}}$ shadows coincide |
| state trajectory | coarse STAB/MAGIC state classes | empirical class-transition observable | skew signal $0 \to 0.084$; off-diagonal support $0 \to 2$ |

The first realization is blind to the T/S diagonal-phase distinction. The second converts magic-state production into complete off-diagonal support in a two-state coarse graining. STAB and MAGIC are nonlinear state classes rather than orthogonal subspaces of the qubit Hilbert space, so the second construction is a trajectory-induced diagnostic analogue unless a separate finite Markov adapter supplies and validates its own state space and projectors.

Changing observables and changing sectors are not the same operation. One changes what counts as a part of the report; the other changes what the report can detect. Every realization must declare its sector origin, observable construction, truncation, evaluator, thresholds, and failure modes. Multiple realizations are allowed; hidden realization choices are not.

### 8.2 Attention-Derived Candidate Partitions

A central application domain is neural-system diagnosis. Transformer-style systems already contain mechanisms that partition information: activation patterns, attention heads, token groups, residual-stream directions, and expert-routing decisions [13, 5].

The Qwen audit shows that attention heads provide **data-dependent candidate coordinate partitions**. Such a partition may seed a strict SOF reconstruction only after the retained finite space, complete projectors, and operative matrices are explicitly bound and validated. Attention grouping by itself is not strict sector admission.

The Qwen attention audit (Artifact B7) uses the revision-pinned `Qwen/Qwen2.5-0.5B-Instruct` model [11]. The recorded configuration audits layer 12 and groups tokens by their strongest attention target. For the 45-token probe, the layer has 14 heads. The observed head diversity is:

The reference artifact records the exact model/tokenizer commit, Transformers and PyTorch versions, device, dtype, input text, layer, head, and filtering parameters.

| Head | Groups | Interpretation |
|---|---|---|
| Head 3 | 1 group | global attention: all tokens attend to one target |
| Head 1 | 3 groups, sizes `[38,4,3]` | coarse three-way token clustering |
| Head 6 | 9 target groups, 4 groups after filtering groups of size at least 2 | candidate coordinate partition used for the retained token-space audit |
| Head 13 | 13 groups | dispersed attention partition |

Thus, one pretrained layer supplies multiple candidate partition granularities: global, coarse, intermediate, and dispersed. SOFRS does not impose these partitions or admit them automatically; it records them and asks what observable shadows they carry.

Using the Head 6 sectors together with the layer's attention matrices motivates a strict reconstruction, but the frozen envelope records only derived shadows and matrix-family descriptors. The migration therefore keeps both the support and commutator outputs on an analogue carrier. The main significance is structural: attention heads provide candidate finite coordinate partitions at different resolutions before compiler emission and report assembly.

### 8.3 Behavioral Walls

Prompt collections do not define wall geometry. Instruction conflict, refusal, schema collapse, context saturation, or prompt injection becomes a candidate observable wall only after a typed chart, one-parameter path, comparison map, observable, norm, and threshold are declared; otherwise it remains an analogue trajectory descriptor.

Likewise, few-shot repair, instruction tuning, and preference optimization [10] are protocol-level behavioral observations, not Lie-depth repair or claims about hidden mechanisms.

---

## 9 Case Studies: Compiler-Profile-Selected Reports

### 9.1 Three Bounded Controls

| Control | Retained observation | Admission boundary |
|---|---|---|
| Qwen attention | a 45-token probe yields a 40-dimensional retained space, a Head 6 candidate partition, and a 75.0% off-diagonal support shadow | the frozen envelope does not bind the restricted matrices as source-addressed $(V, Q, Y)$; the record remains analogue, `strict_reconstruction=yes` denotes bounded enumerability only, and commutator depth remains a general-matrix proxy |
| Dynamic maze | connected-component sectors change from 1 to 25 and back, with split and merge descriptors | the aggregate is a schema transition rather than one fixed typed chart; pointwise strict records would require separately validated projectors and alphabets |
| API-only LLM | six prompt protocols across three task classes are evaluated deterministically | protocol classes do not become operator blocks, routed products, words, Lie repair, or hidden mechanisms |

The commonality is the compiler interface and evidence discipline, not a claim that the three mechanisms or output modules are identical. Pairwise comparison still requires the alignment object of Paper XIII.

---

## 10 Versioned Protocol Migration and Cross-Domain Controls

Migration is one operation of the Versioned Reporting Protocol, not a fourth compiler contract or an independent paper-level pillar. The first migration tests whether frozen v1 reports can be admitted into SOFRS v2.0 without changing their source artifacts or silently promoting their typed claims.

### 10.1 Version Boundary

SOFRS v1.0 remains immutable. It required one eight-field envelope containing `Sectorization`, `Observable Family`, `Support Matrix`, `Bridge Matrix`, `Repair Matrix`, `Wall Record`, `Claim Status`, and `Failure Modes`. That format was useful for disclosure but allowed unlike carriers to share a field.

SOFRS v2.0 does not reinterpret a v1 field in place. A versioned adapter reads the frozen artifact, records its digest, declares capabilities, normalizes legacy sentinels and semantic labels, and emits new Manifest, IR, and report artifacts. Thus

$$\text{v1 envelope validity} \not\Longrightarrow \text{v2 strict admission.}$$

The frozen v1 schema and validators remain executable. The v2.0 schema, profiles, migration index, and validator are separate versioned artifacts.

SOFRS v2.1 is a second, explicit boundary-annotation migration. A v2.0 report does not validate directly as v2.1. Source-addressed migration preserves its normative projection, adds revision provenance and the fixed `object_transition_boundary`, and requires a new v2.1 receipt. This is a semantic revision, not a claim of wire backward compatibility.

### 10.2 Registry of Migrated Reports

The v2 migration is a contract and semantic audit of the nine frozen v1 reports. It does not recompute the underlying experiments. Each v1 artifact and its producer are retained with SHA-256 digests and translated by one versioned adapter.

| Migrated report | v2 record kind | Enabled strict/analogue content | Main correction |
| --- | --- | --- | --- |
| Transformer activation | `diagnostic_analogue` | finite activation descriptors | explicit operative matrices are not bound by the frozen envelope |
| Transformer batch sweep | `diagnostic_analogue` | cross-configuration robustness descriptor | changing ambient dimensions prevent one strict record |
| Qwen attention | `diagnostic_analogue` | support and commutator-proxy descriptors | explicit operative matrices are not bound by the frozen envelope |
| MoE route sectors | `diagnostic_analogue` | route and positive-word descriptors | the routing operator is not bound as an explicit reconstruction artifact |
| MoE bias repair | `diagnostic_analogue` | routing activation descriptors | no explicit complete $(V, Q, Y)$ was declared |
| Diffusion trajectory | `diagnostic_analogue` | sampled schema-transition descriptor | pointwise sectors do not imply one moving chart |
| Dynamic maze | `diagnostic_analogue` | component split/merge descriptor | varying component projectors are a schema transition |
| Recommender coverage | `diagnostic_analogue` | coverage and cutoff-depth descriptors | before/after operative matrices are not bound as reconstruction artifacts |
| API-only LLM | `diagnostic_analogue` | behavioral descriptor module | probe classes are not projector sectors |

**Migration/Assembly Certificate (Migration Census).**
The source-addressed migration index records nine inputs, nine `diagnostic_analogue` outputs, four bounded reconstruction assessments with status `yes`, and 118 typed sentinel replacements. Its `claim_target=migration_consistency` and `certificate_class=migration_assembly` establish migration consistency only, not source-experiment recomputation or adapter adequacy.

---

## 11 Failure Modes and Applicability

The multi-system **validator fixture** (Artifact B14) contains five constructed structural boundary cases. It remains a frozen v1 envelope fixture, excluded from v2 migration and ordinary protocol admission. An API infrastructure boundary is listed separately because it may occur within one analogue report:

| Case | Case interpretation | Diagnostic reason |
| --- | --- | --- |
| Single sector | cross-sector modules not applicable | support, bridge, and repair have no cross-sector pair; global one-sector findings may remain reportable |
| Dense random all-to-all observables | no contrast | immediate full support destroys structure |
| Over-refined one-dimensional sectors | over-refined | sectorization is too fine to expose subspace structure |
| Commuting matrices | empty commutator proxy | the declared matrices commute; no Lie claim is made without a Lie/Hall carrier |
| Sector-observable mismatch | probe mismatch | observables do not see the proposed sector interface |
| API infrastructure failure | infrastructure failure | provider or backend errors dominate the measurement |

These interpretations are not claim-status values. The combined fixture is not one source realization and therefore is not a normal v2 report.

Protocol rejection applies to carrier or policy substitution, source-digest drift, claim/finding disagreement, analogue-to-strict masquerading, invalid item bindings, and unsatisfied external-basis references. A validator's `PASS` assertion is not self-authenticating.

A scientifically misleading but internally consistent adapter presents a different challenge. No schema can identify such a failure from syntax alone. Without a domain baseline or falsifying source evidence, the protocol must leave adequacy unresolved and withhold an Object Certificate rather than claim automatic rejection. Conversely, with an external baseline, a contradicted adapter is rejected for the object claim even if its report remains schema-valid.

The admission checks are now typed:

1. strict admission requires finite complex $(V, Q, Y)$ data and a structural certificate;
2. analogue admission requires provenance, descriptors, an analogue mapping, and a negative strict boundary;
3. every enabled module requires its own carrier, convention, policy, and evidence contract;
4. dense, trivial, or mismatched data may pass structural validation while remaining scientifically uninformative;
5. provider failure must be separated from model behavior;
6. unavailable capabilities must not be replaced by nearby carriers.

---

## 12 Machine-Readable Protocol Boundary

This paper requires a reproducible path from source artifacts through the Capability Manifest, Typed IR, selected Compiler Report Profile, bound `CompilerOutput`, Assembly Profile, and assembled report. Cross-file references are digest-bound; no particular software API or command-line interface is required.

The protocol ends at report production and validation. Alignment and normalized comparison belong to Paper XIII; Paper XIV ends at policy-relative interpretation and bounded candidates. Selection, authorization, and execution require further downstream contracts.

Protocol validation recomputes compilation and assembly, checks item identity and report equality, and verifies contract shape, references, evidence links, profile gates, strict/analogue exclusion, and migration rules. It does not decide domain adequacy.

The resulting receipt binds the checked report closure and validator contract. For a v2.0-to-v2.1 migration it also cites the frozen v2.0 receipt. The receipt closure is ordered and digest-checked, but its `PASS` field is not self-authenticating. It certifies protocol conformance only, not adapter adequacy, alignment, interpretation, or action.

---

## 13 Claim Spine

Definitions and negative ownership boundaries are not additional evidence levels. The reader-facing status map is:

| Claim or object | Formal role and claim target | Reader-facing status |
|---|---|---|
| SOFRS v2.1 Report Protocol and Assembly Profiles | owned normative definitions; representation interface | not an independent evidence claim |
| Assembly Faithfulness | exact protocol invariant; implementation checked by a Protocol Conformance Certificate | Theorem |
| Report Non-Intervention | exact protocol invariant from the declared type signatures | Theorem |
| Report Relativity | exact representation-interface proposition under the protocol definition | Theorem |
| Claim-Scoped External Constraint Registry (`external_basis_registry`) | source, object, structure, and domain-basis evidence routing | not an independent evidence claim |
| Adapter Adequacy Boundary | epistemic principle and negative boundary; no certificate promotion | not an independent evidence claim |
| Object-level adapter result | independently recomputed finite source fact with a satisfied external basis (Object Certificate) | Computational Certificate |
| Migration census: `9/9/4/118` | finite executable migration audit (Migration/Assembly Certificate) | Computational Certificate |
| Qwen, maze, API, and quantum controls | bounded source-addressed controls | Computational Observation |
| canonical realization and realization invariance | open uniqueness targets; no certificate is claimed here | Research Program |

---

## 14 Boundary

This paper owns single-report assembly and the SOFRS protocol boundary. It inherits Paper X compilation, preserves each normative `CompilerOutput` item exactly once, and does not perform alignment, interpretation, or selection.

This paper does not claim:

1. existence, uniqueness, or superiority of a realization over native domain methods;
2. promotion of proxy or analogue observations to strict carriers without an additional realization or bridge theorem;
3. that protocol validity establishes adapter adequacy, alignment, interpretation, authorization, or action;
4. that protocol non-intervention proves arbitrary implementations are side-effect free.

---

## 15 Conclusion

SOFRS v2.1 assembles one bound `CompilerOutput` into a capability-gated, realization-relative report. It preserves compiler items, keeps strict and analogue carriers separate, and leaves source adequacy to external domain evidence. Comparison and interpretation remain the separately owned operations of Papers XIII and XIV.

---

# Appendix A Normative SOFRS v2.1 Contract

The normative v2.1 contract is identified by source digest rather than duplicated in full in this manuscript. Artifact B21 defines the assembled report envelope. Artifacts B22–B23 are strict and analogue Compiler Report Profile instances under the Paper X Compiler Report Profile v1.0 contract. Artifacts B29–B31 define the distinct Paper XII Assembly Profile contract and instances. The Capability Manifest, Typed SOF IR, Compiler Report Profile, and derivation rule schemas are the versioned Paper X compiler contracts indexed by Artifact B24.

A v2.1 report requires the following grouped fields:

| Group | Required fields |
|---|---|
| Identity | sofrs_version, report_id, system, record_kind |
| Revision and boundary | revision_provenance, object_transition_boundary, strict_reconstruction, external_basis_registry |
| Contracts | compiler_contracts, compiler_output_binding, assembly_contract |
| Content and bindings | item_bindings, alignment_readiness, source_mapping, source_artifacts |
| Results and provenance | modules, findings, claims, degradation_items, failure_modes, provenance |

provenance is a disjoint union: native_generation binds the native source, adapter, compiler output, assembly profile, and producer closure; migration binds a non-native SOFRS v1 source, the migration adapter/ruleset, and its receipt. A native report cannot carry the migration variant.

The required boundary is constant:

| Field | Required value |
|---|---|
| artifact_role | INFORMATIONAL_REPORT |
| intervention_semantics | NONE |
| source_state_transition_authority | NONE |
| implementation_purity_status | NOT_ESTABLISHED_BY_SOFRS |

It states that compilation, assembly, serialization, and protocol validation have artifact codomains and no intervention authority over the source state. It does not prove that an arbitrary implementation is side-effect free.

The v2.1 receipt fixes receipt_kind = SOFRS_VALIDATION_RECEIPT and receipt_scope = SOFRS_PROTOCOL_CONFORMANCE_ONLY. Its PASS status certifies the checked report closure only; it carries no intervention semantics and no implementation-purity claim.

It does not require universal support, bridge, repair, or wall fields. The admission constraints are:

| Record kind | Admission constraint |
|---|---|
| strict_sof | An enabled sof-basic module is required. |
| diagnostic_analogue | An enabled diagnostic-analogue module is required; strict-SOF theorem claims are forbidden. |

Every affirmative statement must reference an admitted IR claim or finding and retain its carrier, convention, policy, evidence, scope, and derivation state. Every normative claim or degradation item must also retain its source_output_item_id; the binding list must be a typed bijection with the bound CompilerOutput.items array. failure_modes does not encode compiler degradation. Missing capabilities produce module omission or an explicit unavailable statement. All contract and source references carry 64-hex SHA-256 digests.

### A.1 Claim-Scoped External Basis Mapping

The semantic external basis is serialized by `external_basis_registry`. Its four levels and mandatory constraint identifiers are:

- **Source identity:** `source_identity`, bound to `source-snapshot-pinned`, records identity and digest closure for the cited source snapshot.
- **Object recomputation:** `object_level`, bound to `object-level-recomputation`, records independent evidence for the cited finite object fact.
- **Realization/structure validation:** `structure_level`, bound to `realization-structure-validation`, records satisfaction of the strict structural admission basis.
- **Semantic adequacy:** `semantic_adequacy`, bound to `domain-semantic-adequacy`, records external assessment of domain relevance and baseline adequacy.

Packages and constraints use `SATISFIED`, `PARTIAL`, `NOT_ASSESSED`, or `NOT_APPLICABLE`. Every `SATISFIED` entry carries source-addressed evidence. Each claim binds its own package and constraint subset through `external_basis_refs` and `external_constraint_ids`; report-level validity does not transfer one claim's satisfied basis to another. `basis_status=COMPLETE` is reserved for registries whose applicable packages and mandatory constraints are all satisfied.

An Object Certificate requires a cited, satisfied `object_level` package and independently checkable evidence. A `strict_sof` report requires a satisfied `structure_level` package. Otherwise the artifact may retain only its admitted protocol/representation claim or an unresolved external basis. These checks govern classification and admission; they do not establish scientific adequacy by schema validity alone.

For a `strict_sof` report, `strict_reconstruction.candidate_status` is `not_applicable`: the candidate predicate applies only before strict admission.

---

## Appendix B SOFRS v2.1 Example Reports

The migrated collection supplies three compact protocol patterns:

| Pattern | Example | Required presentation |
|---|---|---|
| strict-reconstruction boundary | Qwen attention | analogue descriptors, producer provenance, missing explicit $(V, Q, Y)$ boundary, and controlled reconstruction assessment |
| diagnostic analogue | API-only LLM | descriptor provenance, evaluator outputs, analogue mapping, and negative strict boundary |
| graceful degradation | dynamic maze aggregate | schema-transition findings and explicit unavailability of fixed-chart strict fields |

These examples instantiate the protocol; they do not enlarge the Paper X compiler contracts or authorize promotion between their carriers.

---

## Appendix C Frozen SOFRS v1.0 Provenance Contract

This subsection preserves the frozen compatibility contract. A SOFRS v1.0 artifact contains the version key `sofrs_version` and the eight diagnostic fields defined by the frozen schema. The controlled `claim_status` vocabulary supports machine-readable aggregation and prepares reports for later aligned comparison, while `claim_note` carries non-normative human qualification. Domain-specific metadata may be added without changing the eight-field report grammar. The executable schema is Artifact B1. It is not the normative contract for new v2 reports.

The frozen JSON Schema defines envelope validity only. Protocol admission additionally requires `report_id`, `system`, `claim_note`, an explicit failure boundary, and conditional provenance for the historical Level III behavioral regime. Those rules are encoded by Artifact B2; they do not mutate the frozen v1.0 envelope contract. Downstream protocol keys such as `reference`, `candidate`, `alignment`, `signature`, `comparison_role`, `transformation_contract`, `contract_evaluation`, `action_semantics`, `action_set`, and `selection` are reserved for downstream comparison and action artifacts and should not appear as top-level SOFRS fields.

Artifact B1 is the source-addressed frozen v1 schema. Artifact B3 validates the envelope against that schema, and Artifact B4 applies the separate v1 admission profile. The executable contract, rather than a copy in this manuscript, is the source of truth.

The schema forbids the superseded top-level names `schema_version`, `wall_records`, `repair_candidates`, and `trajectory_summary`.

A trajectory summary, when present, is nested inside `wall_record`. Specification revisions that change required keys or controlled vocabularies must increment the SOFRS version rather than silently changing the meaning of v1.0.

---

## Appendix D v1-to-v2 Migration Mapping and Certificate

The migration is non-destructive and source-addressed:

| v1 element | v2 destination | Migration rule |
|---|---|---|
| one eight-field envelope | Manifest + IR + output + report | split declaration, compiler emission, and presentation |
| support/bridge/repair field | carrier-qualified finding or analogue descriptor | no global text substitution |
| `999` depth sentinel | `UNREACHED_AT_CUTOFF` | require the inherited cutoff policy |
| report-level claim label | result state + claim status | enforce their legal pairing |
| implicit missing field | unavailable module | do not serialize absence as zero |
| source file | source artifact reference | pin path and SHA-256 digest |
| external source basis | `external_basis_registry` and claim refs | separate source/object/structure/adequacy; do not infer |

The executable certificate consists of the historical migration index, nine migrated Manifest/IR/`CompilerOutput`/v2.0 report stacks and receipts, and the explicit v2.0-to-v2.1 migration with nine new reports and receipts. The v2.1 validator reproduces the boundary, projection, closure, and receipt-role checks. The index records nine analogue reports, four controlled strict-reconstruction assessments with status `yes`, and 118 sentinel normalizations. These counts certify the migrated collection only; they do not claim that every historical v1 field has a strict v2 equivalent.

---

# Appendix E Computational Artifacts

This appendix indexes the executable artifacts used for contract validation and the reported controls. The historical B-series artifact identifiers are retained for source-address stability. Paths are relative to the directories stated below.

## E.1 Contracts and Validators

The default directory in this table is `schemas/sofrs/` for B1–B2, B21–B24, and B28–B31, and `experiments/paper12/validation/` for B3–B4, B32–B34.

| Artifact | Role in this paper | Short path |
|---|---|---|
| B1 | v1 envelope schema | `v1.0.schema.json` |
| B2 | v1 admission profile | `paper12-protocol-profile-v1.0.json` |
| B3 | v1 schema-drift validator | `validate_sofreport.py` |
| B4 | v1 admission validator | `validate_protocol_admission.py` |
| B21 | v2.1 report schema | `v2.1.schema.json` |
| B22 | strict Compiler Report Profile v1.0 instance | `paper12-strict-compiler-profile-v1.0.json` |
| B23 | analogue Compiler Report Profile v1.0 instance | `paper12-analogue-compiler-profile-v1.0.json` |
| B24 | compiler contracts and rule registry | `../sofcompiler/` |
| B28 | v2.1 validation-receipt schema | `report-validation-receipt-v2.1.schema.json` |
| B29 | Assembly Profile v2.0 schema | `assembly-profile-v2.0.schema.json` |
| B30 | strict Assembly Profile instance | `paper12-strict-assembly-profile-v2.0.json` |
| B31 | analogue Assembly Profile instance | `paper12-analogue-assembly-profile-v2.0.json` |
| B32 | canonical normative-core and artifact-identity helper | `canonical_identity.py` |
| B33 | v2.0-to-v2.1 non-intervention migrator | `migrate_sofrs_v2_to_v2_1.py` |
| B34 | v2.1 semantic and receipt validator | `validate_sofrs_v2_1.py` |

## E.2 Report-Protocol Audits

The default directory in this table is `experiments/paper12/`.

| Artifact | Role in this paper | Short path |
|---|---|---|
| B5 | activation-sector audit | `transformer_activation_sof.py` |
| B6 | transformer batch sweep | `transformer_batch_sweep.py` |
| B7 | Qwen attention audit | `qwen_attention_sof.py` |
| B8 | MoE route-sector audit | `moe_expert_sof.py` |
| B9 | private-expert bias control | `moe_bias_repair_sof.py` |
| B10 | diffusion aggregate | `diffusion_denoising_sof.py` |
| B11 | maze aggregate | `maze_wall_crossing.py` |
| B12 | recommender audit | `recommender_sof.py` |
| B13 | API analogue audit | `blackbox_llm_sof.py` |
| B14 | boundary-fixture generator | `failure_cases.py` |
| B15 | frozen v1 reports | `archive/results/*.sofreport` |
| B16 | v1 validator fixture | `archive/results/failure_cases.fixture.json` |
| B25 | v1-to-v2 adapter | `validation/migrate_sofrs_v1_to_v2.py` |
| B26 | frozen v2 validator and receipt checker | `validation/validate_sofrs_v2.py` |
| B27 | v2 stack and validation receipts | `results/` |
| B35 | v2.1 reports and conformance-only receipts | `results/v2.1/` |

### E.3 Cross-Program Support

The default directory in this table is `experiments/`.

| Artifact | Role in this paper | Short path |
|---|---|---|
| B17 | gate-log T-blind control | `quantum/quantum_gateset_t_blind_control.py` |
| B18 | state-trajectory probe | `quantum/quantum_state_trajectory_sof.py` |
| B19 | control/PDE handoff | `paper10/control_pde_combinatorial_sof.py` |
| B20 | finance handoff | `paper10/barrier_option_sof.py` |

---

The B-series artifacts certify only their declared compiler, assembly, migration, identity, or bounded control targets. They do not establish adapter adequacy, report alignment, interpretation, or action. Imported cross-program support retains its owning paper and claim status. The complete artifact map and runnable contract checks are indexed in `experiments/paper12/README.md`; all listed artifacts are available in the RIME repository.

## References


[1] WuJun Chen. Capability-aware compilation for sectorized observable frameworks: Typed admission and cross-species registry evidence. *Preprint*, 2026. Paper X of the RIME program, version 2.0. DOI: `https://doi.org/10.5281/zenodo.21768257`.

[2] WuJun Chen. Observable dynamics of sectorized observable frameworks: Typed deformations, observable trajectories, and wall pullbacks. *Preprint*, 2026. Paper IX of the RIME program, version 2.1. DOI: `https://doi.org/10.5281/zenodo.21991080`.

[3] WuJun Chen. Sectorized observable framework: A typed static object language for sectorized observables. *Preprint*, 2026. Paper VIII of the RIME program, version 2.1. DOI: `https://doi.org/10.5281/zenodo.21977464`.

[4] WuJun Chen. Typed wall morphology for sectorized observable frameworks: Sparse wall records, coordinate profiles, multi-label taxonomy, and local-model eligibility. *Preprint*, 2026. Paper XI of the RIME program, version 2.0. DOI: `https://doi.org/10.5281/zenodo.21801722`.

[5] Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning. What does BERT look at? an analysis of BERT's attention, 2019. arXiv:1906.04341.

[6] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, et al. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models, 2024. arXiv:2401.06066.

[7] DeepSeek-AI. DeepSeek-V3 technical report, 2024. arXiv:2412.19437.

[8] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models, 2020. arXiv:2006.11239.

[9] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229, 2019.

[10] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback, 2022. arXiv:2203.02155.

[11] Qwen Team. Qwen2.5 technical report, 2024. arXiv:2412.15115.

[12] Inioluwa Deborah Raji, Andrew Smart, Rebecca N. White, Margaret Mitchell, Timnit Gebru, Ben Hutchinson, Jamila Smith-Loud, Daniel Theron, and Parker Barnes. Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, pages 33–44, 2020.

[13] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.